

LAB 6. MODEL DEPLOYMENT WITH ONNX

EXPORT ONNX, CONSISTENCY TEST AND BENCHMARK

ThS. Lê Thị Thùy Trang

2026-04-27

1 Giới thiệu bài thực hành

1.1 Mục tiêu bài thực hành

Sau các bài lab trước, sinh viên đã làm quen với việc huấn luyện mô hình học sâu và theo dõi thí nghiệm. Tuy nhiên, trong một hệ thống thật, câu chuyện không dừng lại ở việc mô hình đạt accuracy tốt trên tập test. Một mô hình muốn được sử dụng trong sản phẩm cần được đóng gói, kiểm thử và đo tốc độ suy luận một cách cẩn thận.

Trong bài lab này, chúng ta chuyển từ tư duy **train model** sang tư duy **deploy model**. Mô hình đã được huấn luyện từ bài trước sẽ được export từ PyTorch sang ONNX. Sau đó, sinh viên cần kiểm tra xem mô hình ONNX có cho kết quả gần giống mô hình PyTorch hay không. Cuối cùng, sinh viên đo latency, throughput và kích thước mô hình để có cơ sở nhận xét về khả năng triển khai.

Nói nôm na, bài này trả lời câu hỏi rất thực tế:

Mô hình chạy được trong notebook rồi, nhưng nếu đem ra triển khai thì có ổn không?

Trong bài lab này, sinh viên cần:

- Hiểu vai trò của ONNX trong triển khai mô hình học sâu;
- Export được checkpoint PyTorch sang file `.onnx`;
- Chạy inference bằng `onnxruntime`;
- Kiểm tra consistency giữa output PyTorch và output ONNX;
- Biết vì sao cần dùng `model.eval()` khi export và inference;
- Biết vì sao benchmark cần có warm-up và lặp nhiều lần;
- Đo được latency, throughput và kích thước mô hình;
- Đọc kết quả benchmark để đưa ra nhận xét có căn cứ.

1.2 Kết quả đầu ra mong đợi của bài lab

Sau bài lab này, sinh viên cần có:

1. Một repo chạy được cho quy trình export ONNX và benchmark;
2. Một file ONNX export từ checkpoint PyTorch, ví dụ `vit_smartcampus.onnx`;
3. Một file `export_report.json` ghi lại thông tin export;
4. Một file `consistency_report.json` kiểm tra độ nhất quán PyTorch–ONNX;

5. Một file `benchmark_results.csv` chứa kết quả benchmark theo batch size;
6. Một file `benchmark_summary.json` tổng hợp thông tin benchmark;
7. Một báo cáo ngắn giải thích kết quả, không chỉ dán số liệu;
8. Nhận xét được khi nào ONNXRuntime nhanh hơn, khi nào chưa chắc nhanh hơn;
9. Rút ra bài học về triển khai mô hình thay vì chỉ huấn luyện mô hình.

1.3 Những thứ bắt buộc phải nộp

Mỗi sinh viên cần nộp tối thiểu:

1. Repo code chạy được.
2. README hướng dẫn chạy lại các bước export, consistency test và benchmark.
3. File `export_report.json`.
4. File `consistency_report.json`.
5. File `benchmark_results.csv`.
6. File `benchmark_summary.json`.
7. Báo cáo theo mẫu `REPORT_TEMPLATE.md` hoặc báo cáo riêng.
8. Nhận xét ngắn: ONNXRuntime có nhanh hơn PyTorch không, và vì sao có thể như vậy.
9. Nhận xét ngắn: consistency test có pass không, nếu không pass thì nghi ngờ lỗi ở đâu.
10. Link lưu checkpoint/file ONNX nếu file quá lớn và không commit trực tiếp lên GitHub.

Lưu ý quan trọng: không commit dataset, checkpoint lớn hoặc file ONNX quá lớn lên GitHub. Nếu GitHub cảnh báo file lớn, sinh viên cần lưu file ở Google Drive, OneDrive hoặc nguồn lưu trữ phù hợp, sau đó ghi link trong báo cáo.

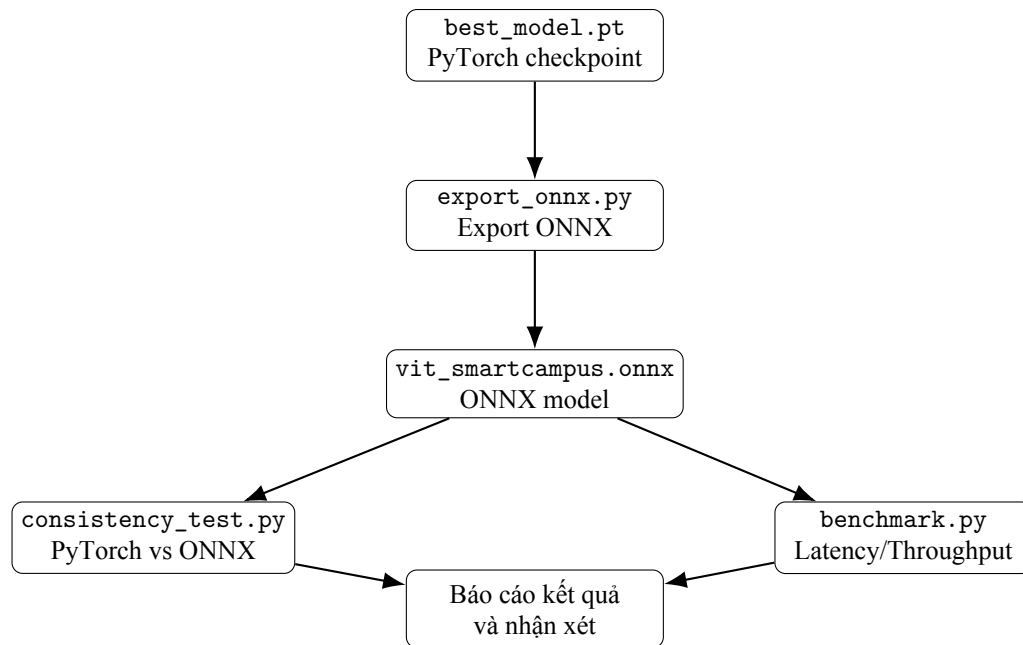
1.4 Bối cảnh bài toán

Chúng ta tiếp tục đóng vai nhóm kỹ sư AI xây dựng một mô-đun nhỏ cho hệ thống Smart Campus. Ở bài trước, mô hình Vision Transformer đã được huấn luyện để phân loại ảnh không gian trong trường học. Mỗi ảnh đầu vào có thể thuộc một trong năm lớp:

- `classroom`;
- `computerroom`;
- `library`;
- `corridor`;
- `office`.

Trong bài lab này, ta giả sử mô hình PyTorch đã train xong và được lưu dưới dạng `best_model.pt`. Bây giờ, nhiệm vụ không phải là cố train thêm cho accuracy tăng một chút, mà là kiểm tra xem mô hình có thể bước ra khỏi môi trường huấn luyện hay chưa.

Luồng xử lý chính của bài lab như sau:



Nếu bài lab trước giống như xây một chiếc xe, thì bài lab này giống như đưa xe ra đường thử: máy có nổ không, đi có đúng hướng không, tốc độ có ổn không, và có đáng tin để giao cho người khác sử dụng không.

1.5 Cấu trúc repo starter kit

Repo starter kit của bài lab có cấu trúc như sau:

```

csc4005_lab6_onnx_consistency_benchmark_starter/
  README.md
  REPORT_TEMPLATE.md
  requirements.txt
  configs/
    export_vit_onnx.json
    consistency_test.json
    benchmark_cpu.json
  docs/
    LAB_GUIDE_LAB6_ONNX.md
    ONNX_GUIDE.md
    RUBRIC.md
    TROUBLESHOOTING.md
  notebooks/
    lab6_onnx_demo.ipynb
  src/
    dataset.py
    model.py
    runtime.py
    export_onnx.py
    infer_pytorch.py
    infer_onnx.py
    consistency_test.py
    benchmark.py
    utils.py
  ci/

```

```
check_structure.py
smoke_export.py
checkpoints/
outputs/
.github/workflows/
ci.yml
```

Trong đó, các file quan trọng nhất là:

- `src/export_onnx.py`: export checkpoint PyTorch sang ONNX;
- `src/consistency_test.py`: so sánh output PyTorch và output ONNX;
- `src/benchmark.py`: đo latency, throughput và kích thước mô hình;
- `REPORT_TEMPLATE.md`: mẫu báo cáo sinh viên cần điền;
- `configs/`: lưu các cấu hình mẫu để sinh viên dễ chạy lại.

2 Chuẩn bị môi trường thực hành

2.1 Điều kiện cần

Sinh viên cần chuẩn bị:

- Python 3.10 hoặc phiên bản tương thích;
- Git;
- Một repo GitHub Classroom hoặc repo cá nhân;
- Checkpoint `best_model.pt` từ lab ViT trước đó;
- Subset dữ liệu MIT Indoor Scenes 67 gồm 5 lớp đã dùng ở lab trước;
- Máy tính có đủ RAM để chạy inference với ViT.

Bài lab này có thể chạy trên CPU. Nếu có GPU thì tốt hơn, nhưng benchmark CPU vẫn là một tình huống triển khai rất có ý nghĩa, vì nhiều hệ thống inference thực tế không phải lúc nào cũng có GPU.

2.2 Clone repo

Sau khi nhận link bài tập từ GitHub Classroom, sinh viên clone repo về máy:

```
git clone <student-repo-url>
cd csc4005_lab6_onnx_consistency_benchmark_starter
```

Kiểm tra nhanh cấu trúc repo:

```
python ci/check_structure.py
```

Nếu lệnh này báo `Structure check passed.` thì cấu trúc cơ bản đã ổn.

2.3 Khởi tạo environment

Trên macOS hoặc Linux:

```
python3 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
```

Trên Windows:

```
python -m venv .venv
.venv\Scripts\activate
pip install --upgrade pip
```

2.4 Cài thư viện phục vụ bài lab

Cài các thư viện cần thiết:

```
pip install -r requirements.txt
```

Các thư viện quan trọng gồm:

- torch, torchvision: dựng lại mô hình PyTorch;
- onnx: kiểm tra file ONNX sau export;
- onnxruntime: chạy inference bằng ONNXRuntime;
- numpy, pandas: xử lý số liệu benchmark;
- Pillow: đọc ảnh;
- tqdm: hiển thị tiến trình.

2.5 Chuẩn bị checkpoint

Sinh viên cần đặt checkpoint từ lab trước vào thư mục:

```
checkpoints/best_model.pt
```

Checkpoint này nên là checkpoint đã train cho bài toán Smart Campus Scene Classification. Nếu checkpoint được lưu ở nơi khác, sinh viên có thể truyền đường dẫn trực tiếp vào tham số `--checkpoint`.

Ví dụ:

```
python -m src.export_onnx \
--checkpoint /duong_dan/dan/best_model.pt \
--onnx_path outputs/vit_smartcampus.onnx \
--dynamic_batch
```

2.6 Chuẩn bị dữ liệu kiểm thử

Dữ liệu dùng cho consistency test nên có cấu trúc như sau:

```
data/mit_indoor_smartcampus_5/
classroom/
computerroom/
library/
corridor/
office/
```

Nếu dữ liệu quá lớn, sinh viên không cần đặt toàn bộ vào repo. Có thể đặt dữ liệu ở ngoài thư mục repo và truyền đường dẫn bằng tham số `--data_dir`.

Ví dụ:

```
python -m src.consistency_test \
  --checkpoint checkpoints/best_model.pt \
  --onnx_path outputs/vit_smartcampus.onnx \
  --data_dir /duong_dan/mit_indoor_smartcampus_5
```

2.7 Nhắc lại quy tắc không push file lớn lên GitHub

Các thư mục sau đã được đưa vào `.gitignore`:

```
data/
checkpoints/
outputs/
*.pt
*.pth
*.onnx
```

Sinh viên cần lưu ý: GitHub không phải nơi phù hợp để lưu dataset hoặc checkpoint lớn. Repo GitHub nên chứa code, hướng dẫn chạy, báo cáo và các file kết quả nhỏ. File lớn thì lưu ở nơi khác và ghi link trong báo cáo.

3 Voila, hướng dẫn thực hành ở đây

3.1 Kiểm tra pipeline bằng smoke export

Trước khi chạy với checkpoint thật, sinh viên có thể kiểm tra pipeline export bằng mô hình khởi tạo ngẫu nhiên. Lệnh này không cần checkpoint và không dùng để nộp kết quả chính.

```
python -m src.export_onnx \
  --onnx_path outputs/debug_random_vit.onnx \
  --model_name vit_b_16 \
  --num_classes 5 \
  --img_size 224 \
  --opset 17 \
  --dynamic_batch \
  --no_checkpoint
```

Nếu chạy thành công, repo sẽ tạo ra:

```
outputs/debug_random_vit.onnx
outputs/export_report.json
```

Smoke export giống như kiểm tra đường ống nước trước khi bơm nước thật. Nếu đường ống còn rò rỉ thì chưa nên vội đem checkpoint thật vào chạy.

3.2 Bước 1. Export mô hình PyTorch sang ONNX

Khi đã có checkpoint thật, chạy lệnh sau:

```
python -m src.export_onnx \
--checkpoint checkpoints/best_model.pt \
--onnx_path outputs/vit_smartcampus.onnx \
--model_name vit_b_16 \
--img_size 224 \
--opset 17 \
--dynamic_batch
```

Sau khi chạy, kiểm tra thư mục `outputs/`:

```
outputs/
vit_smartcampus.onnx
export_report.json
```

Mở file `export_report.json`, sinh viên cần thấy các thông tin như:

```
{
  "onnx_path": "outputs/vit_smartcampus.onnx",
  "model_name": "vit_b_16",
  "num_classes": 5,
  "img_size": 224,
  "opset": 17,
  "dynamic_batch": true,
  "onnx_size_mb": 300.0,
  "status": "exported_and_checked"
}
```

Con số `onnx_size_mb` có thể khác nhau tùy mô hình. Không cần hoảng nếu file ONNX khá lớn, vì ViT-base vốn không phải mô hình nhỏ.

3.3 Vì sao phải có `model.eval()`?

Khi export hoặc inference, mô hình cần được đưa về chế độ đánh giá bằng:

```
model.eval()
```

Nếu quên bước này, một số lớp như dropout có thể hoạt động theo kiểu huấn luyện, làm output thay đổi giữa các lần chạy. Khi đó, consistency test có thể fail dù code export nhìn qua có vẻ không sai.

Nói cách khác:

Export được file ONNX chưa chắc đã đúng. Phải kiểm tra output của nó.

3.4 Vì sao nên dùng `dynamic batch`?

Khi export, nếu không khai báo `dynamic batch`, file ONNX có thể chỉ nhận đúng batch size lúc export, ví dụ batch size bằng 1. Điều này không thuận tiện khi triển khai, vì trong thực tế ta có thể muốn chạy batch size 1, 4, 8 hoặc nhiều hơn.

Trong bài lab này, tham số:

```
--dynamic_batch
```

được dùng để cho phép batch size thay đổi.

3.5 Bước 2. Chạy inference thử bằng PyTorch và ONNX

Sinh viên có thể chọn một ảnh bất kỳ trong dataset để chạy thử inference.

Ví dụ chạy PyTorch:

```
python -m src.infer_pytorch \  
--checkpoint checkpoints/best_model.pt \  
--image_path data/mit_indoor_smartcampus_5/classroom/xxx.jpg \  
--model_name vit_b_16 \  
--img_size 224
```

Ví dụ chạy ONNXRuntime:

```
python -m src.infer_onnx \  
--onnx_path outputs/vit_smartcampus.onnx \  
--image_path data/mit_indoor_smartcampus_5/classroom/xxx.jpg \  
--img_size 224
```

Nếu hai mô hình dự đoán cùng lớp thì đó là tín hiệu tốt. Tuy nhiên, kiểm tra một ảnh là chưa đủ. Chúng ta cần consistency test trên nhiều mẫu.

3.6 Bước 3. Consistency test: PyTorch vs ONNX

Chạy lệnh sau:

```
python -m src.consistency_test \  
--checkpoint checkpoints/best_model.pt \  
--onnx_path outputs/vit_smartcampus.onnx \  
--data_dir data/mit_indoor_smartcampus_5 \  
--num_samples 32 \  
--batch_size 8 \  
--atol 1e-4 \  
--rtol 1e-3
```

Sau khi chạy, repo tạo file:

```
outputs/consistency_report.json
```

File này nên chứa các thông tin như:

```
{  
  "num_samples": 32,  
  "batch_size": 8,  
  "max_abs_diff": 0.00003,  
  "mean_abs_diff": 0.000001,  
  "pred_match_rate": 1.0,  
  "atol": 0.0001,  
  "rtol": 0.001,  
  "passed": true  
}
```

3.7 Đọc consistency_report.json thế nào?

Các trường quan trọng nhất là:

- `max_abs_diff`: sai khác tuyệt đối lớn nhất giữa logits của PyTorch và ONNX;
- `mean_abs_diff`: sai khác trung bình;
- `pred_match_rate`: tỷ lệ mẫu mà PyTorch và ONNX dự đoán cùng nhãn;
- `passed`: bài kiểm thử có vượt ngưỡng hay không.

Nếu `max_abs_diff` hơi khác 0 nhưng `pred_match_rate` bằng 1.0 thì thường vẫn ổn. Do khác biệt số học giữa runtime, PyTorch và ONNXRuntime không nhất thiết phải giống nhau từng chữ số cuối cùng.

3.8 Khi nào consistency test fail?

Consistency test có thể fail khi:

- checkpoint không khớp với kiến trúc model;
- số lớp khi dựng model khác số lớp lúc train;
- preprocessing ảnh không giống nhau;
- quên `model.eval()`;
- export thiếu dynamic axes hoặc sai input shape;
- dùng nhầm checkpoint từ bài lab khác.

Nếu test fail, không nên vội kết luận ONNX sai. Hãy kiểm tra lại từng mắt xích, đặc biệt là checkpoint, số lớp, input size và preprocessing.

3.9 Bước 4. Benchmark latency, throughput và model size

Sau khi consistency test ổn, sinh viên benchmark:

```
python -m src.benchmark \
--checkpoint checkpoints/best_model.pt \
--onnx_path outputs/vit_smartcampus.onnx \
--data_dir data/mit_indoor_smartcampus_5 \
--batch_sizes 1 4 8 \
--warmup 10 \
--repeat 50
```

Output kỳ vọng:

```
outputs/benchmark_results.csv
outputs/benchmark_summary.json
```

File `benchmark_results.csv` có dạng:

```
runtime,batch_size,mean_latency_ms,median_latency_ms,p95_latency_ms,throughput_img_per_sec,model_size_m
PyTorch,1,...
ONNXRuntime,1,...
PyTorch,4,...
ONNXRuntime,4,...
PyTorch,8,...
ONNXRuntime,8,...
```

3.10 Nếu nhìn bảng benchmark như nhìn bức vách thì không sao hết, đọc hướng dẫn dưới đây là ổn áp ngay

3.10.1 Những cột quan trọng nhất

- `runtime`: đang đo PyTorch hay ONNXRuntime;
- `batch_size`: số ảnh chạy trong một lượt inference;
- `mean_latency_ms`: thời gian trung bình cho một lượt chạy;
- `median_latency_ms`: thời gian trung vị, thường ổn định hơn mean;
- `p95_latency_ms`: 95% lượt chạy nhanh hơn hoặc bằng giá trị này;
- `throughput_img_per_sec`: số ảnh xử lý được trong một giây;
- `model_size_mb`: kích thước file model.

3.10.2 Latency là gì?

Latency là thời gian mô hình xử lý một lượt inference. Nếu batch size bằng 1, latency gần với thời gian phản hồi cho một ảnh.

Ví dụ, nếu `mean_latency_ms` = 80, tức là trung bình mô hình mất khoảng 80 mili-giây cho một lượt inference.

3.10.3 Throughput là gì?

Throughput là số ảnh xử lý được trong một giây. Batch size lớn hơn thường làm throughput tăng, nhưng latency cho cả batch cũng tăng.

Điều này tạo ra trade-off:

- Nếu cần phản hồi nhanh từng ảnh, batch size nhỏ có thể hợp lý;
- Nếu cần xử lý nhiều ảnh offline, batch size lớn có thể tốt hơn;
- Không có batch size tốt nhất cho mọi tình huống.

3.10.4 Vì sao cần warm-up?

Những lần inference đầu tiên thường chậm hơn do runtime cần khởi tạo, cấp phát bộ nhớ hoặc tối ưu graph. Nếu đo luôn từ lần đầu tiên, số liệu có thể bị lệch.

Vì vậy benchmark cần có:

```
--warmup 10
```

Lệnh này giúp bỏ qua một số lượt chạy ban đầu trước khi đo thật.

3.10.5 Vì sao cần repeat nhiều lần?

Nếu chỉ đo một lần, kết quả có thể bị ảnh hưởng bởi tiến trình nền của máy tính, CPU đang bận hoặc hệ điều hành đang làm việc khác.

Vì vậy cần lặp lại nhiều lần:

```
--repeat 50
```

Sau đó ta đọc mean, median và p95 thay vì tin vào một con số đơn lẻ.

3.11 So sánh PyTorch và ONNXRuntime

Sau khi có `benchmark_results.csv`, sinh viên cần lập bảng so sánh trong báo cáo.

Gợi ý bảng:

Runtime	Batch size	Mean latency (ms)	P95 latency (ms)	Throughput (img/s)	Model size (MB)
PyTorch	1	...	...	...	...
ONNXRun- time	1	...	...	...	...
PyTorch	4	...	...	...	...
ONNXRun- time	4	...	...	...	...
PyTorch	8	...	...	...	...
ONNXRun- time	8	...	...	...	...

Câu hỏi cần trả lời:

1. ONNXRuntime có nhanh hơn PyTorch ở batch size 1 không?
2. ONNXRuntime có nhanh hơn PyTorch ở batch size 4 hoặc 8 không?
3. Throughput thay đổi như thế nào khi tăng batch size?
4. P95 latency có chênh lệch nhiều so với mean latency không?
5. Nếu triển khai thật, nên chọn runtime và batch size nào?

3.12 Bước cuối cùng: so bó đũa, chọn cột cờ

Đến cuối bài lab, sinh viên không chỉ cần nói “em export được rồi”, mà cần đưa ra một kết luận kỹ thuật.

Ví dụ kết luận tốt:

Mô hình ONNX đã được export thành công với dynamic batch. Consistency test trên 32 mẫu cho thấy `pred_match_rate = 1.0`, `max_abs_diff` nhỏ hơn ngưỡng đặt ra. Khi benchmark trên CPU, ONNXRuntime nhanh hơn PyTorch ở batch size 1 và 4, nhưng mức chênh lệch không quá lớn. Với hệ thống cần phản hồi từng ảnh, batch size 1 là phù hợp; với xử lý theo lô, batch size 4 cho throughput tốt hơn.

Ví dụ kết luận chưa tốt:

Em thấy ONNX nhanh hơn. Kết quả ở trong file benchmark.

Kết luận chưa tốt vì không nói rõ nhanh hơn ở đâu, bao nhiêu, theo metric nào, và trong điều kiện nào.

4 Khi nào ONNXRuntime tốt hơn PyTorch?

ONNXRuntime thường hữu ích khi ta muốn triển khai mô hình trong môi trường inference gọn hơn, độc lập hơn với code huấn luyện. Tuy nhiên, ONNXRuntime không phải cây đũa thần. Có lúc nó nhanh hơn rõ rệt, có lúc chỉ nhanh

hơn một chút, và có lúc không nhanh hơn nếu mô hình, phần cứng hoặc cấu hình runtime chưa phù hợp.

Vì vậy, trong bài lab này, sinh viên cần học một thái độ quan trọng:

Không đo thì không kết luận.

Một số yếu tố ảnh hưởng đến kết quả benchmark:

- loại mô hình;
- batch size;
- CPU/GPU;
- phiên bản PyTorch, ONNX và ONNXRuntime;
- số thread CPU;
- input size;
- số lần warm-up và repeat.

5 Những lỗi rất hay gặp

5.1 Lỗi 1. Sai đường dẫn checkpoint

Triệu chứng thường gặp:

```
FileNotFoundError: checkpoints/best_model.pt
```

Cách xử lý: kiểm tra checkpoint có thật sự nằm trong thư mục checkpoints/ không. Nếu checkpoint ở nơi khác, truyền đúng đường dẫn qua `--checkpoint`.

5.2 Lỗi 2. Checkpoint không khớp kiến trúc model

Triệu chứng thường gặp là lỗi khi `load_state_dict`, ví dụ thiếu key hoặc sai kích thước layer cuối.

Nguyên nhân có thể là:

- checkpoint được train bằng model khác;
- số lớp lúc export khác số lớp lúc train;
- checkpoint không phải từ lab ViT;
- model name đang để `vit_b_16` nhưng checkpoint lại từ kiến trúc khác.

5.3 Lỗi 3. Quên `model.eval()`

Nếu quên `model.eval()`, output có thể không ổn định. Trong starter kit, bước này đã được xử lý, nhưng sinh viên vẫn cần hiểu vì sao nó quan trọng.

5.4 Lỗi 4. Sai input size

Nếu model được train với ảnh 224x224 nhưng lúc export hoặc inference lại dùng kích thước khác, output có thể lỗi hoặc không như kỳ vọng.

Trong bài lab này, mặc định dùng:

```
--img_size 224
```

5.5 Lỗi 5. Không warm-up khi benchmark

Nếu không warm-up, vài lượt inference đầu tiên có thể làm kết quả latency trông chậm bất thường. Khi đó, bảng benchmark dễ gây hiểu nhầm.

5.6 Lỗi 6. Repeat quá ít

Nếu `--repeat` quá nhỏ, kết quả có thể dao động mạnh. Không nên đo 1–2 lần rồi kết luận mô hình nhanh hay chậm.

5.7 Lỗi 7. Nhầm latency và throughput

Latency thấp nghĩa là một lượt chạy nhanh. Throughput cao nghĩa là xử lý được nhiều ảnh trong một giây. Hai metric này liên quan nhưng không giống nhau.

Một mô hình có thể throughput tốt khi batch size lớn nhưng latency cho từng batch lại tăng. Vì vậy, phải đọc cả hai.

5.8 Lỗi 8. Push file lớn lên GitHub

Nếu sinh viên push `.pt`, `.onnx` hoặc dataset lớn, GitHub có thể báo lỗi. Cách tốt hơn là không commit file lớn, chỉ ghi link lưu trữ trong báo cáo.

6 Câu hỏi tự kiểm tra sau lab

Sinh viên cần tự trả lời các câu hỏi sau trước khi nộp bài:

1. ONNX là gì và khác gì so với checkpoint PyTorch?
2. Vì sao export thành công chưa đủ để khẳng định mô hình ONNX chạy đúng?
3. Consistency test kiểm tra điều gì?
4. `max_abs_diff` và `mean_abs_diff` có ý nghĩa gì?
5. `pred_match_rate` bằng 1.0 có nghĩa là gì?
6. Vì sao cần dùng `model.eval()` trước khi export?
7. Vì sao benchmark cần warm-up?
8. Vì sao benchmark cần repeat nhiều lần?
9. Latency và throughput khác nhau như thế nào?
10. Khi tăng batch size, latency và throughput thường thay đổi như thế nào?
11. Nếu ONNXRuntime không nhanh hơn PyTorch trong thí nghiệm của bạn, điều đó có nghĩa là bài lab sai không?
12. Trong hệ thống Smart Campus thật, ngoài latency còn cần quan tâm những yếu tố nào?

7 Checklist trước khi nộp bài

Trước khi nộp, sinh viên kiểm tra lại:

- Repo có chạy được không?
- Có hướng dẫn chạy trong README hoặc báo cáo không?
- Có file `export_report.json` không?
- Có file `consistency_report.json` không?
- Có file `benchmark_results.csv` không?
- Có file `benchmark_summary.json` không?
- Có giải thích consistency test pass/fail không?
- Có so sánh PyTorch và ONNXRuntime không?
- Có nhận xét về latency, throughput và model size không?
- Có tránh push file lớn lên GitHub không?

Nếu tất cả đều ổn, xin chúc mừng: mô hình của bạn đã đi thêm một bước rất quan trọng từ “mô hình trong notebook” sang “mô hình có thể nghĩ đến chuyện triển khai”.